Number Theory Study Guide

1. Sieve of Eratosthenes

The Sieve of Eratosthenes is an ancient algorithm used to find all prime numbers up to a given limit. It works by iteratively marking the multiples of each prime number starting from 2.

How it works:

1. Create a list of consecutive integers from 2 through n
2. Start with the first number (2)
3. Mark all multiples of 2 (except 2 itself) as composite
4. Move to the next unmarked number and repeat
5. Continue until you've processed all numbers

Java Implementation:

```java
public class SieveOfEratosthenes {
    public static boolean[] sieve(int n) {
        boolean[] isPrime = new boolean[n + 1];

        // Initialize all numbers as prime
        for (int i = 2; i <= n; i++) {
            isPrime[i] = true;
        }

        for (int i = 2; i * i <= n; i++) {
            if (isPrime[i]) {
                // Mark all multiples of i as composite
                for (int j = i * i; j <= n; j += i) {
                    isPrime[j] = false;
                }
            }
        }

        return isPrime;
    }

    public static void printPrimes(int n) {
        boolean[] isPrime = sieve(n);
        System.out.print("Prime numbers up to " + n + ": ");
        for (int i = 2; i <= n; i++) {
            if (isPrime[i]) {
                System.out.print(i + " ");
            }
        }
        System.out.println();
    }
}
```

Time Complexity: O(n log log n)

Space Complexity: O(n)

2. GCD and LCM

Greatest Common Divisor (GCD)

The GCD of two numbers is the largest positive integer that divides both numbers without a remainder.

Euclidean Algorithm:

```java
public class GCDAndLCM {
    // Recursive approach
    public static int gcd(int a, int b) {
        if (b == 0) {
            return a;
        }
        return gcd(b, a % b);
    }

    // Iterative approach
    public static int gcdIterative(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }
}
```

Least Common Multiple (LCM)

The LCM of two numbers is the smallest positive integer that is divisible by both numbers.

Formula: LCM(a, b) = (a × b) / GCD(a, b)

```java
public static int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

// For multiple numbers
public static int lcmOfArray(int[] numbers) {
    int result = numbers[0];
    for (int i = 1; i < numbers.length; i++) {
        result = lcm(result, numbers[i]);
    }
    return result;
}
```

Example Usage:

```java
public static void main(String[] args) {
    int a = 48, b = 18;
    System.out.println("GCD of " + a + " and " + b + " = " + gcd(a, b)); // Output: 6
    System.out.println("LCM of " + a + " and " + b + " = " + lcm(a, b)); // Output: 144
}
```

3. Modular Arithmetic

Modular arithmetic is a system of arithmetic for integers where numbers "wrap around" when reaching a certain value called the modulus.

Basic Operations:

Addition:

```java
public static int modAdd(int a, int b, int mod) {
    return ((a % mod) + (b % mod)) % mod;
}
```

Subtraction:

```java
public static int modSub(int a, int b, int mod) {
    return ((a % mod) - (b % mod) + mod) % mod;
}
```

Multiplication:

```java
public static int modMul(int a, int b, int mod) {
    return ((a % mod) * (b % mod)) % mod;
}
```

Modular Exponentiation

Computing (base^exp) % mod efficiently:

```java
public static long modPow(long base, long exp, long mod) {
    long result = 1;
    base = base % mod;

    while (exp > 0) {
        // If exp is odd, multiply base with result
        if (exp % 2 == 1) {
            result = (result * base) % mod;
        }

        // exp must be even now
        exp = exp / 2;
        base = (base * base) % mod;
    }

    return result;
}
```

Modular Inverse

Finding the multiplicative inverse of a number modulo m using Extended Euclidean Algorithm:

```java
public static int modInverse(int a, int m) {
    int[] result = extendedGCD(a, m);
    int gcd = result[0];
    int x = result[1];

    if (gcd != 1) {
        return -1; // Inverse doesn't exist
    }

    return (x % m + m) % m;
}

public static int[] extendedGCD(int a, int b) {
    if (a == 0) {
        return new int[]{b, 0, 1};
    }

    int[] result = extendedGCD(b % a, a);
    int gcd = result[0];
    int x1 = result[1];
    int y1 = result[2];

    int x = y1 - (b / a) * x1;
    int y = x1;

    return new int[]{gcd, x, y};
}
```

Properties of Modular Arithmetic:

- (a + b) mod m = ((a mod m) + (b mod m)) mod m
- (a - b) mod m = ((a mod m) - (b mod m) + m) mod m
- (a × b) mod m = ((a mod m) × (b mod m)) mod m
- (a^n) mod m can be computed efficiently using modular exponentiation

---

Quiz Questions

1. What is the time complexity of the Sieve of Eratosthenes algorithm? a) O(n) b) O(n log n) c) O(n log log n) d) $O(n^2)$

Answer: c) O(n log log n)

2. What is the GCD of 48 and 18? a) 4 b) 6 c) 8 d) 12

Answer: b) 6

3. If GCD(a, b) = 6 and LCM(a, b) = 144, what is the value of a × b? a) 864 b) 720 c) 900 d) 1008

Answer: a) 864 (because a × b = GCD × LCM = 6 × 144 = 864)

4. What is (23 × 15) mod 7? a) 1 b) 2 c) 3 d) 4

Answer: b) 2 (because 23 × 15 = 345, and 345 mod 7 = 2)

5. In the Sieve of Eratosthenes, why do we only check up to √n? a) To save memory b) Because all composite numbers > √n have already been marked c) It's a programming convention d) To reduce time complexity

Answer: b) Because all composite numbers > √n have already been marked

---

Practice Coding Questions

1. Write a Java method that takes an integer n and returns the count of prime numbers less than or equal to n using the Sieve of Eratosthenes.

2. Implement a method that finds the GCD of three numbers using the Euclidean algorithm.

3. Create a program that calculates (a^b) mod m efficiently for large values of a, b, and m.

4. Write a method that determines if two numbers are coprime (their GCD is 1).

5. Implement a function that finds all prime factors of a given number and returns them as an array.